

CS584: Natural Language Processing Notes

Steven DeFalco

Spring 2024

Contents

1	Regular Expressions, Text Normalization, Edit Distance	2
1.1	Regular Expressions	2
1.1.1	Basic Regular Expression Patterns	2
1.1.2	Disjunction, Grouping, and Precedence	3
1.1.3	More Operators	3
1.2	Words	4
1.3	Text Normalization	4
2	Lecture 2: Machine Learning Basics	5
2.1	Neural Networks	6
3	Deep Feedforward Networks	7
4	Vector Semantics	8
4.1	Word Vectors	8
5	Language Modeling	9
5.1	Neural Language Modeling	10
6	More on RNNs	11
7	CNN, Tokenization	12
7.1	Convolutional Neural Networks	12
7.2	Tokenization	12
8	Machine Translation and Seq2Seq Models	13
9	Attention and Transformers	14
9.1	Self-Attention	15
10	LLMs: Pretraining and Finetuning	16
10.1	Pretraining	16
10.2	Finetuning	17

1 Regular Expressions, Text Normalization, Edit Distance

Regular expressions can be used to specify strings we might want to extract from a document. **Normalizing** text means converting it to a more convenient, standard form. For example, most of what we are going to do with language relies on first separating out or **tokenizing** words from running text. Another part of text normalization is **lemmatization**, the task of determining that two words have the same root, despite their surface differences. For example, the words *sang*, *sung*, and *sings* are forms of the verb *sing*. The word *sing* is the common lemma of these words, and a **lemmatizer** maps from all of these to *sing*. **Stemming** refers to a simpler version of lemmatization in which we mainly just strip suffixes from the end of the word. Text normalization also includes **sentence segmentation**: breaking up a text into individual sentences, using cues like periods or exclamation points. Finally, we'll need to compare words and other strings. We'll introduce a metric called **edit distance** that measures how similar two strings are based on the number of edits (insertions, deletions, substitutions) it takes to change one string into the other.

1.1 Regular Expressions

Formally, a regular expression is an algebraic notation for characterizing a set of strings. Regular expressions are particularly useful for searching in texts, when we have a pattern to search for a **corpus** of texts to search through. A regular expression search function will search through the corpus, returning all texts that match the pattern.

1.1.1 Basic Regular Expression Patterns

The simplest kind of regular expression is a sequence of simple characters; putting characters in sequence is called **concatenation**. Regular expressions are **case sensitive**. We can solve this with the use of square braces [and]. The string of characters inside the braces specifies **disjunction** of characters to match. For example, the pattern `/[wW]/` matches patterns containing either *w* or *W*.

The square braces can also be used to specify what a single character *cannot* be, by use of the caret `^`. If the caret is the first symbol after the open square brace, the resulting pattern is negated. For example, the pattern `/[^a]/` matches any single character (including special characters) except *a*. This is only true when the caret is the first symbol after the open square brace.

We use the question mark `/?/`, which means *the preceding character or nothing*. For example, `/woodchucks?/` will match with either *woodchuck* or *woodchucks*. The **Kleene star** means *zero or more occurrences of the immediately previous*

character or regular expression. So $/a^*/$ means *any string of zero or more as.* *Kleene+* means *one or more occurrences of the immediately preceding character or regular expression.* The period is used as a **wildcard** expression that matches any single character.

1.1.2 Disjunction, Grouping, and Precedence

The **disjunction** operator, also called the **pipe** symbol $|$ matches either the preceding or following strings. Typically, regular expressions always match the *largest* string they can: we say that patterns are **greedy**, expanding to cover as much of a string as they can. There are, however, ways to enforce **non-greedy** matching, using another meaning of the $?$ qualifiers. The operator $*?$ is a Kleene star that matches as little text as possible. The operator $+?$ is a Kleene plus that matches as little text as possible.

1.1.3 More Operators

Below are some aliases for common ranges, which can be used mainly to save typing.

- $/d$: any digit
- $/D$: any non-digit
- $/w$: any alphanumeric/underscore
- $/W$: a non-alphanumeric
- $/s$: whitespace (space, tab)
- $/S$: non-whitespace

A range of numbers can also be specified. So $/\{n,m\}/$ specifies from n to m occurrences of the previous char or expression. REs for counting are summarized below.

- $*$: zero or more occurrences of the previous char or expression
- $+$: one or more occurrences of the previous char or expression
- $?$: zero or one occurrences of the previous char or expression
- $\{n\}$: exactly n occurrences of the previous char or expression
- $\{n,m\}$: from n to m occurrences of the previous char or expression
- $\{n,\}$: at least n occurrences of the previous char or expression
- $\{,m\}$: up to m occurrences of the previous char or expression

Some certain special characters are referred to by special notation based on the backslash. The most common of these are the **newline** character `/ n` and the **tab** character `/ ^`

- `/ *`: an asterisk
- `/ .:` a period
- `/ ?:` a question mark
- `/ n:` a newline
- `/ t:` a tab

1.2 Words

A **lemma** is a set of lexical forms having the same stem, the same major part-of-speech, and the same word sense. The **word form** is the full inflected or derived form of the word. **Types** are the number of distinct words in a corpus; if the set of words in the vocabulary is V , the number of types is the vocabulary size $|V|$. **Tokens** are the total number N of running words.

The larger the corpora we look at, the more word types we find, and in fact this relationship between the number of types $|V|$ and number of tokens N is called **Herdan's Law** or **Heaps' Law**. It is shown below where k and β are positive constants, and $0 < \beta < 1$.

$$|V| = kN^\beta$$

The value of β depends on the corpus size and the genre. Roughly then we can say that the vocabulary size for a text goes up significantly faster than the square root of its length in words.

1.3 Text Normalization

Before almost any natural language processing of a text, the text has to be normalized. At least three tasks are commonly applied as part of any normalization process:

1. Tokenizing (segmenting) words
2. Normalizing word formats
3. Segmenting sentences

One commonly used tokenization standard is known as the **Penn Treebank tokenization** standard. This standard separates out clitics (*doesn't* becomes

does plus *n't*), keeps hyphenated words together and separates out all punctuation.

Most tokenization schemes have two parts: a token learner and a token segmenter. The ***token learner*** takes a raw training corpus and introduces a vocabulary, a set of token. The ***token segmenter*** takes a raw test sentence and segments it into the tokens in the vocabulary. The most widely used algorithm is byte-pair encoding. The BPE token learner begins with a vocabulary that is just the set of all individual characters. It then examines the training corpus, chooses the two symbols that are most frequently adjacent, adds a new merged symbol to the vocabulary, and replaces every adjacent in the corpus with the new symbol. It continues to count and merge, creating new longer and longer character strings, until k merges have been done creating k novel tokens; k is thus a parameter of the algorithm. The resulting vocabulary consists of the original set of characters plus k new symbols. The algorithm is usually run inside words, so the input corpus is first white-space-separated to give a set of strings, each corresponding to the characters of a word, plus a special end-of-word symbols, and its counts.

2 Lecture 2: Machine Learning Basics

The ***training set*** is the sample of data used to fit the model. The ***validation set*** is the sample of data used to provide unbiased evaluation of a model fit on the training dataset while tuning model hyperparameters. The ***test dataset*** is the sample of data used to provide an unbiased evaluation of a final model fit on the training dataset; this cannot be used for training.

Cross-validation allows us to estimate the generalization error based on training examples alone. In ***k-fold cross validation*** the original sample is randomly partitioned in k equal sized subsamples. Of the k subsamples, a single subsample is retained as the validation data for testing the model, and the remaining $k - 1$ subsamples are used as training data.

In ***supervised learning***, we have a training dataset containing samples. In each samples, x_i are inputs and y_i are labels. We try to predict the labels using the inputs, and this is the basis of training. With ***unsupervised learning***, we have no labels and instead expect the model to learn some patterns from the dataset.

For **document classifications**, build a feature vector for each document

$$\{x_i, y_i\}_{i=1}^N$$

where x_i can be word counts, TF-IDF, topic distributions, etc. . .

A ***loss function*** ($\text{Loss}(x, y, w)$) qualifies how wrong you would be if you used w to make a prediction in x when the correct output is y . It is the object we want

to minimize. Loss is a function of the parameters w and we can try to minimize it directly. We reduce the estimation problem to a **minimization** problem.

We have a cost function $J(\theta)$ we want to minimize. **Gradient descent** is an algorithm to minimize $J(\theta)$: for the current value θ , calculate the gradient of $J(\theta)$, then take small step in direction of negative gradient, and repeat. The **gradient** is the direction that increases the loss the most; we want to take the *negative* gradient in gradient descent.

Gradient descent is slow. An **epoch** is one pass through the data. Each iteration requires going over all training examples; this is expensive and slow when we have lots of data. In practice, we can use **stochastic gradient descent**. At each epoch, we can randomly shuffle the data to ensure that each data point creates an *independent* change on the model, without being biased by the same points before them. In this implementation, it may never reach the local minima and could oscillate around it due to the fluctuations in each step. Instead of using the whole data for calculating gradient, in **mini-batch gradient descent** we use only a mini-batch of it instead of the whole dataset.

Learning rate affects the update of gradients; it is the amount that we update the weights. A too large learning rate may lead to diverging values. A too small learning rate may cost a lot of time to convert to the minimum points. **Adaptive learning rate** involves reducing the learning rate by some factor every few epochs. Divide the learning rate of each parameter by the root mean square of its previous derivatives.

2.1 Neural Networks

Logistic regression gives only linear decision boundaries which are limited and unhelpful when a problem is complex. Neural networks can learn much more complex functions and **nonlinear** decision boundaries. Some of the *pros* of neural networks are the following:

- used on a variety of domains
- predictions are very quick

Backpropagation provides an efficient procedure to compute derivatives. Without nonlinearities, deep neural networks can't do anything more than a linear transform. Extra layers could just be compiled into a single linear transform.

When initializing...

- initialize weights to small random values
- initialize hidden layers to 0 and output biases to optimal value if weights were 0

Regularization prevents overfitting when we have a lot of features. A loss function in practice includes regularization over all parameters. **Early stopping** is a popular regularization technique in which we monitor the performance for every epoch on the validation set during the training set and terminate the training as soon as the validation error reaches a minimum.

Dropout provides a computationally inexpensive but powerful method of regularizing a broad family of models. Each neuron has some probability of being dropped out during each iteration.

3 Deep Feedforward Networks

Deep feedforward networks have the goal of approximating some function (target function). They are typically represented by composing together many different functions; there are multiple hidden layers and one output layer.

Nearly all deep learning algorithms can be described as particular instances of a fairly simple recipe: combine

- a specification of a dataset
- a cost function
- an optimization procedure
- and a model

To optimize, for linear models, we could use normal equations, closed form optimization. For nonlinear models, it requires us to choose an iterative numerical optimization procedure. The largest difference between linear models and neural networks is the **nonlinearity** of a neural network causes most interesting loss functions to become **nonconvex**. Thus, neural networks are usually trained by using iterative, gradient-based optimizers that merely drive the cost function to a very low value.

Architecture refers to the overall structure of the network: how many units it should have and how these units should be connected to each other. Most neural networks are organized into groups of units called layers.

A problem is convex if its objective is convex function, the inequality constraints f_j are convex, and the equality constraints h_j are affine.

Momentum is designed to accelerate learning. It accumulates an exponentially decaying moving average of past gradients and continues to move in their direction. Adaptive learning rates are a simple idea where the learning rate is reduced by some factor every few epochs.

4 Vector Semantics

Words can be represented in multiply ways:

- Knowledge-based representation
- One-hot representation
- Co-occurrence Matrix
- Low-dimensional dense word vector

We could represent words as one-hot vectors where the vector dimension is equal to the number of words in the vocabulary. However, this will lead to high dimensional representation and thus inefficiency in storage and computation.

TF-IDF stands for term frequency - inverse document frequency. Where **TF** is the frequency of word v appearing in the current document and **DF** is the number of documents where word v appears.

When representing with co-occurrence matrix there are two options:

- **Window**: use window around each word. Captures both syntactic and semantic information
- **Word-document**: gives general topics

The limitations with simple co-occurrence vectors are the increase in size with vocabulary, very high dimensional (requires a lot of storage), and models are less robust. A solution is to use low dimensional dense vectors.

4.1 Word Vectors

Distributional semantics is the idea that a word's meaning is given by the words that frequently appear close-by. When a word w appears in a text, its **context** is the set of words that appear nearby. Use the many contexts of w to build up a representation of w .

We will build a dense vector for each word, chosen so that the vectors of words that appear in similar context will be similar.

Word2vec is a framework for learning word vectors. The idea is that we have a large corpus of text and every word in a fixed vocabulary is represented by a vector. Go through each position t in the text, which has a center word c and context (outside) words o . Use the similarity of the word vectors for c and o to calculate the probability of o given c (or vice versa). Keep adjusting the word vectors to maximize this probability.

For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t . The objective function J is the (average) negative log likelihood: *minimizing the objective function will result in maximizing the predictive accuracy*. To train a model, we adjust parameters to minimize a loss for a simple convex function over two parameters.

When calculating gradients, find the gradient for each center vector v in a window and also calculate the gradients for outside vectors u . Generally, in each window we will compute updates for all parameters that are being used in that window.

Skip-grams (SG) predict context (outside) words given the center word. Skip-gram model with negative sampling is too computationally expensive. Hence, in standard word2vec you implement the skip-gram model with negative sampling. The main idea is to train a binary logistic regressions for a true/positive pair (center word and word in its context window) versus several noise/negative pairs (the center word paired with a random word). We take k negative samples and maximize the probability that real outside word appears, minimize probability that random words appear around center word.

There are two ways to evaluate word vectors.

- **Intrinsic evaluation** is evaluation on a specific/intermediate subtask. This is fast to compute. Evaluate word vectors by how well their cosine distance after addition captures intuitive semantic and syntactic analogy. Identify the word vector which maximizes the cosine similarity. Using intrinsic evaluation techniques should be handled with care.
- **Extrinsic evaluation** is evaluation on a real task. This can take a long time to compute accurately.

Remark Most words have lots of meanings. One vector can not necessarily capture all these meanings.

Different senses of a word reside in a linear superposition (weighted sum) in standard word embeddings like word2vec. Because of ideas from sparse coding, you can actually separate out the senses (providing they are relatively common).

5 Language Modeling

Language modeling is the task of predicting what word comes next. Formally, given a sequence of words, compute the probability distribution of the next word. A system that does this is called a **language model**. Language modeling is a subcomponent of many NLP tasks.

Pre-training first and then fine-tuning on a specific downstream task, allows to capture context-aware word representation. **BERT** is pretraining with specially

designed tasks on large scale unlabeled data, which is very effective to serve as a general-purpose semantic features and improves the performance of NLP tasks.

An ***n*-gram** is a large chunk of n consecutive words. The idea of n -gram models is to collect statistics about how frequent different n -grams are, and use those to predict the next word. In general, this is an insufficient model of language because language has long-distance dependencies.

The ***Maximum Likelihood Estimate*** (MLE) of some parameter of a model M from a training set T maximizes the likelihood of the training set T given the model M .

Perplexity is the inverse probability of the test set, normalized by the number of words. Minimizing perplexity is the same as maximizing probability. N -grams only work well for word prediction if the test corpus looks like the training corpus. In real life, it often does and thus we must train robust models that generalize. With ***laplace smoothing***, just pretend that we saw each word one more time than we did (just add one to all the counts). ***Backoff*** simply uses less context. ***Interpolation*** mixes the probability estimates from all classes.

In ***Good Turing estimation***, reallocate the probability mass of n -grams that occur $c+1$ times in the training data to the n -grams that occur c times. In particular, reallocate the probability mass of n -grams that were seen once to the n -grams that were never seen.

5.1 Neural Language Modeling

The idea of ***Recurrent Neural Network*** (RNN) is to apply the same weights W repeatedly. RNN can process any length input and model size doesn't increase for longer input. Computation for step t can use information from many steps back and the weights are shared across time stamps. However, recurrent computation is very slow. To **train a RNN language model**...

1. Get a big corpus of text which is a sequence of words.
2. Feed into RNN-LM; compute output distribution for every step t .
3. Loss function on step t is usual cross-entropy between out predicted probability and the true next word.
4. Average this to get the overall loss for entire training set.

Computing loss and gradients across entire corpus is *too expensive*! Stochastic gradient descent allows us to compute loss and gradient for small set of data and update. In practice, consider the inputs as a sentence and compute loss $J(\theta)$ for a sentence, update weights, and repeat. Must backpropagate over time steps $i = t, \dots, 0$, summing gradients as you go ("***backpropagation through time***"). Just like a n -gram LM, we can use a RNN LM to generate text by

repeated sampling. Sampled output is the next step's input.

The standard evaluation metric for language models is **perplexity**. Perplexity is the inverse probability of corpus, according to LM normalized by the number of words. This is equal to the exponential of the cross entropy loss $J(\theta)$.

6 More on RNNs

Vanishing gradient is a problem because gradient signals from far away are lost because it's much smaller than gradient signal from close-by. If the gradient becomes vanishingly small over long distances, then we can't tell whether there's no dependency between step t and $t + n$ in the data and we have wrong parameters to capture the true dependency between t and $t + n$. Due to vanishing gradient, RNN-LMs are better at learning from **sequential recency** than syntactic recency.

If the gradient becomes too big (*exploding gradient problem*), then the SGD update step becomes too big. This can cause bad updates: we take too large a step and reach a bad parameter configuration. A solution is **gradient clipping**: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update (take a step in the same direction, but a smaller step).

The main problem with the vanishing gradient problem is that it's too difficult for the RNN to learn to preserve information over many timesteps. In **Long Short-Term Memory (LSTM)**, on step t , there is a hidden state $h^{(t)}$ and a cell state $c^{(t)}$. Both are vectors of length n . The cell stores long-term information. The LSTM can erase, write, and read information from the cell. The selection of which information is erased/written/read is controlled by three corresponding gates. The gates are also vectors of length n . On each timestamp, each element of the gates can be open (1), closed (0), or somewhere in between.

Forget gate controls what is kept vs forgotten, from previous cell state. **Input gate** controls what parts of the new cell content are written to cell. **Output gate** controls what parts of cell are output to hidden states. **New cell content** is the new content to be written to the cell. **Cell content** erases some content from the last cell state, and writes from new cell content. **Hidden state** reads some content from the cell.

Gated Recurrent Units (GRUs) have two gates. The **update gate** controls what part of hidden states are updated vs preserved. The **reset gate** controls what parts of previous hidden states are used to compute new content. Like LSTM, GRU makes it easier to retain long term information.

The biggest difference between LSTM and GRU is that GRU is quicker to compute and has fewer parameters. LSTM is a good default choice, but switch to GRU if you want something more efficient.

Remark Bidirectional RNNs are only applicable if you have access to the entire input sentence.

RNNs are already *deep* in one dimension, but we can also make them deep in another dimension by applying multiple RNNs—this is a multi-layer RNN. This allows the network to compute more complex representations. High performing RNNs are often multi-layer (but not as deep as convolutional networks).

7 CNN, Tokenization

7.1 Convolutional Neural Networks

CNNs are specialized for applications in computer vision where they can accept images as raw input and build up a hierarchy of features.

Batch normalization is often used in CNNs to transform the convolution output of a batch by scaling the activations to have zero mean and unit variance.

7.2 Tokenization

Tokenization is the first step in any NLP pipeline. It has an important effect on the rest of your pipeline. A ***tokenizer*** breaks unstructured data and natural language text into chunks of information that can be considered as **discrete elements**. Many NLP models assume the input text has been tokenized and encoded as **numerical vectors**.

- **Character tokenization:** the simplest tokenization scheme is to feed each character individually to the model
- **Word tokenization:** split into words and map each word to an integer.
- **Subword tokenization:** combine the best aspects of character and word tokenization

potential problem of a large vocabulary is that it requires neural networks to have an enormous number of parameters; expensive to train, and larger models are more difficult to maintain. A common approach is to limit the vocabulary and discard rare words by considering, say, the most 100,000 common words in the corpus.

The idea with subword tokenization is to combine the best aspects of character and word tokenization. We want to split rare words into smaller units to allow the model to deal with complex words and misspellings. We also want to keep frequent words as unique entities so that we can keep the length of our inputs to a manageable size. One of the tools used is breaking off affixes. Because prefixes, suffixes, and infixes change the inherent meaning of words, they can also

help programs understand a word's function. This can be especially valuable for out of vocabulary words, as identifying an affix can give a program additional insight into how unknown words function. The **sub word** model will search for these sub words and break down words that include them into distinct parts.

Byte pair encoding (BPE) is a simple, effective strategy for defining a subword vocabulary.

1. Start with a vocabulary containing only characters and an *end of word* symbol.
2. Using a corpus of text, find the most common adjacent characters, then add as a subword.
3. Replace instances of the character pair with the new subword; repeat until desired vocabulary size.

All pretrained models use some kind of subword tokenization with a **tuned vocabulary**; usually between 50k and 250k pieces.

8 Machine Translation and Seq2Seq Models

Alignment is the correspondence between particular words in the translated sentence pair. Some words have no counterpart. Alignment can be many-to-one or one-to-many or many-to-many. We learn $P(f, a | e)$ as a combination of many factors, including:

- Probability of particular words aligning
- Probability of particular words have particular fertility

Neural machine translation uses a single end-to-end neural network where both the input and output are sentences. Seq2seq models are end-to-end models made up of two RNNs; they are often referred to as encoder-decoder models. The **decoder** is a text generator. This is different from a simple text generator because the initial states are determined by the encoder.

Encoding of the source sentence provides initial hidden state for the Decoder RNN. The **Encoder RNN** produces an encoding of the source sentence. The **Decoder RNN** is a language model that generates target sentence, conditioned on encoding.

The seq2seq model is an example of a **conditional language model**: language model because the decoder is predicting the next word of the target sentence y and conditional because its predictions are also conditioned on the source sentence x .

In **greedy decoding**, usually we decode until the model produces a ¡END_i token. In **beam search decoding**, different hypotheses may produce ¡END_i tokens on different timestamps; when a hypothesis produces ¡END_i that hypothesis is complete; place it aside and continue exploring other hypotheses via beam search. A small beam size k will be very close to greedy decoding which may result in ungrammatical, unnatural, nonsensical, and incorrect results. A larger k means you consider more hypotheses and thus reduce some of the problems which correspond to a small k .

Advantages of NMT is that it has better performance compared to SMT; a single neural network can be optimized end-to-end; and it requires much less human engineering. Disadvantages of NMT is that it is less interpretable and is difficult to control.

Machine translation can be evaluated using **BLEU** (Bilingual Evaluation Understudy) which has n-gram precision plus a penalty for too-short system translations; higher BLEU scores indicate better translation quality. BLEU is useful of imperfect because good translation can still get a poor BLEU score.

9 Attention and Transformers

Attention treats each word's representation as a query to access and incorporate information from a set of values. Attention provides a solution to the bottle neck problem. The core idea of attention is that on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence.

Use the attention distribution to take a weighted sum of the encoder hidden states. The attention output mostly contains information from the hidden states that received high attention. Then concatenate the attention output with decoder hidden state, then use to compute $\hat{y}$ as before. Sometimes we take the attention output from the previous step, and also feed it into the decoder.

Writing **attention** in equations, we have encoder hidden states $h_1, \dots, h_N \in \mathbb{R}^h$. On timestep t , we have decoder hidden state $s_t \in \mathbb{R}^h$. We get the attention scores e^t for this step:

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

We take softmax to get the attention distribution α^t for this step

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

We use α^t to take a weighted sum of the encoder hidden states to get the attention output a_t

$$a_t = \sum_{i=1}^N \alpha_i^t h_i \in \mathbb{R}^h$$

Finally, we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

$$[a_t; s_t] \in \mathbb{R}^{2h}$$

Attention significantly improves NMT performance; decoder focuses on certain parts of the source. Attention solves the bottleneck problem; decoder looks directly at source; bypass bottleneck. Attention helps with vanishing gradient problem; provides shortcut to far away states. Attention provides some interpretability.

Definition 9.1 Given a set of vector values, and a vector query, attention is a technique to compute a weighted sum for the values, dependent on the query.

There are several ways you can compute $e \in \mathbb{R}^N$ from $h_1, \dots, h_N \in \mathbb{R}^{d_1}$ and $s \in \mathbb{R}^{d_2}$.

- Basic dot-product attention: $e_i = s^T h_i \in \mathbb{R}$
- Multiplicative attention: $e_i = s^T W h_i \in \mathbb{R}$ where $W \in \mathbb{R}^{d_2 \times d_1}$ is a weight matrix
- Additive attention: $e_i = v^T \tanh(W_1 h_i + W_2 s) \in \mathbb{R}$ where $W_1 \in \mathbb{R}^{d_4 \times d_1}$, $W_2 \in \mathbb{R}^{d_4 \times d_2}$ are weight matrices and $v \in \mathbb{R}^{d_4}$ is a weight vector.

9.1 Self-Attention

Every word is both a key and a query simultaneously. Let $w_{i:n}$ be a sequence of words in vocabulary V . For each w_i , let $x_i = E w_i$, where $E \in \mathbb{R}^{d \times |V|}$ is an embedding matrix.

1. Transform each word embedding with weight matrices.

$$q_i = W^Q x_i$$

$$k_i = W^K x_i$$

$$v_i = W^V x_i$$

2. Compute pairwise similarities between keys and queries; normalize with softmax

$$e_{ij} = \text{inner_product}(q_i, k_j) \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

3. Compute output for each word as weighted sum of values

$$o_i = \sum_j \alpha_{ij} v_j$$

10 LLMs: Pretraining and Finetuning

The key idea is to use a pretrained dataset that is somewhat related to your finetuning data so that the pretrained model will have some good representations for the finetuning task as well.

10.1 Pretraining

In modern NLP, all parameters in NLP networks are initialized via pretraining. Pretraining methods hide parts of the input from the model, and train the model to reconstruct those parts. Pretraining can improve NLP applications by serving as parameter initialization. The first step is to pretrain using lots of data to learn general things, and the second step is to finetune using not many labels with the goal of adapting the model to the task.

Pretraining establishes the basis of the abilities of LLMs. LLMs can acquire essential language understanding and generation skills. The scale and quality of the pre-training corpus are critical for LLMs to attain powerful capabilities. Model architectures, acceleration methods, and optimization techniques need to be well designed.

LLMs rely on the pretraining corpus and how it has been preprocessed. Existing LLMs mainly leverage a mixture of diverse public textual datasets as the pre-training corpus. There are two types of pre-training corpora:

- **General data:** with large, diverse, and accessible nature to enhance the language modeling and generalization abilities of language models
- **Specialized data:** endowing LLMs with specific task-solving capabilities

Before pretraining, it is important to remove noisy, redundant, irrelevant, and potentially toxic data.

- Quality filtering
- De-duplication
- Privacy reduction (remove personally identifiable information)
- Tokenization

There are three types of architectures that we could pretrain:

- **Encoders** get bidirectional context. The idea is to replace some fraction of words in the input with a special mask token; then predict these words. Only add loss terms from words that are masked out. (e.g. BERT)

- **Encoder-decoders.** The encoder portion benefits from bidirectional context. The decoder portion is used to train the whole model through language modeling. To pretrain, replace different-length spans from the input with unique placeholders; decode out the spans that were removed. (e.g. BART)
- **Decoders.** We can ignore what they were trained to model and we can finetune them by training a classifier on the last word's hidden state. It's natural to pretrain decoders as language models and then use them as generators. (e.g. GPT)

Remark Pretraining is expensive and impractical on a single GPU. Finetuning is practical and common on a single GPU

Pretrained decoders are good at generating sequences. However, BERT and other pretrained encoders don't naturally lead to nice autoregressive (1-word-at-a-time) generation methods.

BART pre-trains a model combining Bidirectional and Auto-Regressive Transformers. A denoising autoencoder for pretraining sequence-to-sequence models. BART is trained by corrupting text with an arbitrary noising function and learning a model to reconstruct the original text. BART uses a standard transformer-based neural machine translation architecture.

Very large language models seem to perform some kind of learning without gradient steps simply from examples you provide within their contexts. The *in-context* examples seem to specify the task to be performed, and the conditional distribution mocks performing the task to a certain extent. There are three approaches to in-context learning

- **Few-shot:** includes several examples in the prompt to demonstrate the expected answer format and content.
- **One-shot:** only a single example is provided in the prompt to demonstrate the desired task.
- **Zero-shot:** no examples are provided to the model during the generation call. Instead, only the task or request is given as input.

10.2 Finetuning

Full finetuning means we update all the parameters in the network during the finetuning process. **Parameter-efficient finetuning** trains using a few existing or new parameters. Parameter-efficient finetuning aims to reduce the number of trainable parameters while retaining good performance. There are four parameter-efficient finetuning methods for transformer language models:

1. **Adapter tuning** incorporates small neural network modules (called adapter) into each transformer layer. During finetuning, the adapter modules would

be optimized according to the specific task goals, freeze the parameters of the original language.

2. ***Prefix tuning*** adds a prefix of parameters, and freezes all pretrained parameters. The prefix is processed by the model just like real words would be. Prepends a sequence of prefixes, which are a set of trainable continuous vectors, to each Transformer layer in language models.
3. ***Prompt tuning*** incorporates trainable prompt vectors at the input layer. Augments the input text by including a group of soft prompt tokens, then takes the prompt-augmented input to solve specific downstream tasks. During training, only the prompt embeddings would be learned according to task-specific supervisions.
4. ***Low-Rank Adaptation (LoRa)*** learns low-rank "diff" between the pre-trained and finetuned weight matrices. This is easier to learn than prefix-tuning.